ONE-MONTH JUNIOR ENGINEER ASSIGNMENT

Smartphone-Based Autonomous Driving Perception & Sensor Fusion

Duration: 4 weeks / 20 working days Compute: PC (Python-first) Input: smartphone camera + IMU + optional GPS

Mission. Build a reproducible research prototype that receives real-time smartphone video and inertial measurements on a PC, detects road lanes and traffic participants, estimates relative scene depth, and fuses sensor information to improve motion-aware interpretation. VIO/SLAM is a supporting module, not the main deliverable.

System architecture

SMARTPHONE	PC INGEST	PERCEPTION	FUSION / OUTPUT
RGB camera + timestamps Gyro + accelerometer Optional GNSS	Receive + synchronize Record / replay Calibrate	Lanes / road geometry Vehicles + obstacles Monocular depth	Ego-motion (IMU / VIO) Object association / distance Overlay + metrics / alerts

Definition of done (end of Month 1)

• Live phone-to-PC stream with video and at least gyroscope + accelerometer; timestamped logging and deterministic replay.

• Road-facing visual overlay showing lane geometry, tracked road users, and a depth visualization in one timeline.

• Explicit distinction between relative monocular depth and validated metric distance; warnings must not imply unverified absolute accuracy.

• IMU-derived motion visualization and one comparison between vision-only and motion-assisted temporal outputs; VIO integration only if feasible.

• Evidence: Git repository, run instructions, recorded example, short report, benchmark CSV, and demo video.

Safety/scope: research and offline/recorded or passenger-only observations. Do not control steering/braking, operate a phone while driving, or claim the prototype is roadworthy.

Four-week milestones

Week	Primary focus	Required milestone
1	Sensor acquisition + streaming + synchronization	PC receives, logs and replays phone camera/IMU
2	Lane detection + objects/tracking	Side-by-side baselines, overlays and failure analysis
3	Depth + calibration + fusion foundations	Depth-to-object association; IMU/ego-motion baseline
4	Integration + evaluation + demonstration	Unified runtime, quantitative tests, report and video

WEEK 1 — Collect and stream smartphone data to the PC

Outcome: stable and timestamped multi-sensor ingest. Smartphone runs acquisition only; all model inference runs on the PC.

Day 1 — Research sensor and streaming options

• List available phone sensors, sample rates, coordinate axes, operating-system APIs, and privacy/permissions.

• Compare Android Camera2 + SensorManager or iOS AVFoundation + Core Motion; examine practical off-the-shelf acquisition tools and custom app route.

• Compare USB vs Wi-Fi; frame transport (MJPEG / RTSP / WebRTC) and telemetry transport (UDP / WebSocket / TCP). Choose one justified stack.

Day 2 — Implement sensor telemetry

• Send gyroscope (rad/s), accelerometer (m/s²), and available GNSS at a measured rate.

• Define message schema: session_id, sensor_type, monotonic timestamp_ns, phone frame axes, values, units, quality / accuracy, sequence_id.

• Implement PC receiver, live plots, dropped-message counters and CSV/Parquet logging.

Day 3 — Implement camera streaming

• Transmit RGB video to PC with frame index and capture timestamp; measure capture-to-display latency.

• Use a fixed camera resolution/frame rate initially (e.g., 720p/30 fps if sustainable); preserve raw frames when feasible.

• Build viewer with connection status and recording controls.

Day 4 — Synchronize and calibrate

• Research clock domains, phone monotonic clock, network transit delay, clock offset and drift.

• Pair each video frame with nearby IMU samples; document interpolation vs nearest-neighbor and expected error.

• Research camera intrinsics/distortion and phone-to-vehicle mounting/extrinsics; run first checkerboard calibration if practical.

Day 5 — Stress test and document

• Test indoor, stationary, walking/handheld, and safely collected road-facing sequences.

• Report achieved FPS, IMU Hz, latency percentile, jitter, dropped frames and data gaps.

• Deliver runnable streamer + PC logger + 2-minute replay + one-page design decision note.

Gate W1: every saved frame has capture time; IMU measurements can be aligned on the PC. A fast but untimestamped demo does not pass.

Research questions (Week 1)

• What do gyroscope and accelerometer measure physically, and why is raw accelerometer data not simply vehicle acceleration?

• How do we represent phone axes versus camera axes versus vehicle/world axes? What happens if the phone mount rotates?

• Why do rolling shutter, clock skew, transport buffering, and unsynchronized streams damage sensor fusion?

WEEK 2 — Core road perception: lanes, vehicles, obstacles

Outcome: two independent, measurable computer vision baselines running on recorded phone footage before pursuing fusion.

Day 6 — Understand road image geometry

• Research perspective projection, focal length, distortion, homography/bird’s-eye view, vanishing point and lane curvature.

• Study classical lane pipeline: color/gradient masks → ROI → Hough / polynomial fit; note where shadows, rain and worn paint break it.

• Define lane representations: left/right boundaries, centerline, confidence and optional ego-lane region.

Day 7 — Build lane detection baseline

• Prototype classical method; compare to a pretrained lane segmentation / lane-specific network.

• Benchmark curve continuity, missing lanes, false markings, day/night sensitivity and inference time.

• Render centerline / lane corridor and lane-departure indicator only when confidence is sufficient.

Day 8 — Research and run road-user detection

• Study object detection: bounding boxes, confidence, NMS, precision/recall, IoU, mAP.

• Run a lightweight pretrained detector on car, truck, bus, motorcycle, bicycle and person classes; inspect class mapping.

• Implement confidence thresholds and record false positives / small distant misses.

Day 9 — Tracking and motion association

• Study SORT / ByteTrack concepts, Kalman motion model, Hungarian association, track ID persistence and occlusion.

• Add tracking to detections; show stable IDs and trajectories in image coordinates.

• Explain why box-center displacement can reflect ego-motion as well as object movement.

Day 10 — Compare performance

• Run pipelines separately and together on at least three distinct road scenes.

• Measure per-module runtime and total FPS; create failure-case gallery.

• Deliver lane overlay + object tracks + concise model-selection evidence.

Gate W2: lane and object overlays operate on the same recorded sequence with stable frame indices; model outputs are stored independently.

Research questions (Week 2)

• What is the difference between semantic segmentation, instance segmentation, lane detection, and object detection?

• Why can apparent proximity in an image mislead us about true distance?

• When is tracking useful beyond frame-by-frame detection, and how does ego-motion affect the association step?

WEEK 3 — Depth estimation and multi-sensor foundations

Outcome: attach usable depth information to detections and obtain an ego-motion/sensor baseline without mistaking either for ground truth.

Day 11 — Research monocular depth

• Learn pinhole projection, perspective, relative-depth ambiguity, metric-depth training assumptions, and depth evaluation metrics.

• Compare a lightweight Depth Anything V2 model or appropriate alternative to a metric-depth model; note speed/VRAM differences.

• Decide whether output is inverse depth, arbitrary scale, or meters; document it explicitly.

Day 12 — Integrate depth on PC

• Run model on same RGB frames; visualize depth heatmap with colorbar / clearly stated units.

• Project vehicle/pedestrian boxes into depth map; select robust object-depth statistic (e.g. median of interior region) and note background leakage.

• Test latency and temporal depth flicker; do not present estimated relative depth as certified meters.

Day 13 — IMU and Kalman filter concepts

• Study rotation matrices / quaternions, body/world transform, gyroscope integration, gravity, accelerometer bias and drift.

• Study Bayes filtering: predict / update; Kalman filter assumptions; EKF nonlinear measurement model and covariance.

• Build 1D toy Kalman filter with plotted estimate vs noisy observations; then process real gyroscope data and display orientation changes.

Day 14 — Visual odometry / VIO / SLAM research

• Distinguish VO (relative camera motion), VIO (camera + IMU), and SLAM (localization + mapping / loop closure).

• Review feature tracking, epipolar geometry, scale ambiguity, IMU preintegration and calibration requirements.

• Run a prebuilt VO or VIO dataset demo if practical; ORB-SLAM3 integration with custom phone data is stretch work, not a required blocker.

Day 15 — First fusion experiment

• Align IMU yaw changes with apparent lane direction / optical flow on controlled data; plot correlations and sign conventions.

• Compare object tracking stability / lane smoothness with and without simple motion-aware compensation or filtering.

• Document where such compensation fails; deliver depth-associated object tracks and IMU trajectory/orientation plots.

Gate W3: depth overlays and per-object depth estimates exist; timestamped IMU plots are integrated and limitations are written down.

Minimal mathematics juniors must explain

• Camera projection: $x \approx fX/Z$; why image motion and bounding-box size alone do not determine unique metric depth.

• Gyroscope: rotation over time from angular velocity; why drift accumulates.

• Kalman filter: $x_k|_{k-1} = F\,x_{k-1}$, $P_k|_{k-1} = F\,P\,F^T + Q$; $K = P\,H^T(H\,P\,H^T + R)^{-1}$. Define each term.

• Time-to-collision concept and its restrictive assumptions; do not use a naive proxy as a safety-grade collision warning.

WEEK 4 — Fuse, evaluate, and demo

Outcome: integrated PC application that can process a synchronized session with independently testable modules.

Day 16 — Design fusion interfaces

• Specify typed output contracts: Frame, IMUSample, LaneEstimate, Detection, Track, DepthEstimate, EgoMotionEstimate.

• Use one monotonic capture-time base; queue/backpressure strategy; maintain confidence and uncertainty fields.

• Keep learned visual models, tracking, depth and motion as replaceable components.

Day 17 — Integrate the perception stack

• Build one pipeline that runs lane extraction + road user detection/tracking + depth on timestamp-aligned video.

• Associate per-track depth with timestamps; use orientation cues for temporal smoothing, not as a substitute for metric calibration.

• Add graceful degradation (missing IMU, low light, model overload); overlay confidence and stale-data warnings.

Day 18 — Validate with controlled sequences

• Prepare at least: straight road, curved road, traffic with occlusion, weak lane markings; use safely pre-recorded/public videos.

• Report lane continuity, manual label agreement on a sampled set, detection precision/recall or annotated-error examples, ID switches and depth consistency.

• Compare with/without IMU-assisted filtering; report difference even if there is no improvement.

Day 19 — Optimize and package

• Profile preprocessing, detector, lanes, depth, tracking and rendering separately; avoid claiming FPS from detector alone.

• Apply frame scheduling (e.g. depth less frequently than detection) with timestamps and measured staleness.

• Add requirements, configuration, reproducibility, sample session, start scripts and automatic test / benchmark procedure.

Day 20 — Final demonstration and presentation

• Demonstrate phone → PC live ingest and replay → lane + objects + depth + orientation overlay.

• Show 3 successes, 3 failures, metrics and a module latency budget; explain uncertainty and untested operating conditions.

• Submit code, technical report (5–8 pages), 3–5 minute demo video and roadmap for Month 2.

Gate W4: reproducible end-to-end PC demonstration. Functional modules and honest error reporting take priority over a visually impressive but unmeasured demo.

Priority and time management

• Must: sensor stream, time alignment, lane detection, road-user detection/tracking, depth, integrated visualization and metrics.

• Should: IMU orientation and motion-aware temporal filtering; calibrated camera; basic coordinate frames.

• Stretch: full VIO/SLAM with phone camera+IMU, metric object distance validation, BEV occupancy and time-to-collision estimation.

Recommended software organization

phone_client/ (camera + sensors) pc_app/ (ingest, sync, record, replay) perception/ (lane, detector, tracker, depth) localization/ (imu, filtering, optional_vio) fusion/ (alignment, associations, confidence) visualization/ evaluation/ configs/ tests/

Research references to investigate

- Ultralytics detection/tracking: https://docs.ultralytics.com/modes/track/
- Depth Anything V2 (reference implementation): https://github.com/DepthAnything/Depth-Anything-V2
- ORB-SLAM3 (optional advanced VIO research): https://github.com/UZ-SLAMLab/ORB_SLAM3
- OpenCV camera calibration: https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html
- Android motion sensors: https://developer.android.com/develop/sensors-and-location/sensors/sensors_motion
- Apple Core Motion: https://developer.apple.com/documentation/coremotion

Review available licenses, phone platform compatibility, and model hardware requirements before implementation. Referenced tools are candidates, not mandatory architecture choices.

Suggested measurable targets (engineering goals, not guarantees)

- Sensor receive: sustained video around 15–30 fps if device/network allow; IMU sample rate recorded and reasonably stable (e.g., 50–100 Hz).
- Runtime: report actual combined end-to-end FPS, p50/p95 camera-to-overlay latency, queue size, dropped samples and stale age for every module.
- Detection/tracking: manually label a small validation sample; report class-specific misses/false alarms, stability and ID switches.
- Lanes: manually review curves, worn markings, shadows and intersections; report when lane boundary or centerline is unavailable.
- Depth: label outputs as relative unless independently metric-calibrated and evaluated against reference distances.

Final submission checklist

- Git repository with README and one-command PC replay; acquisition app/setup guide.
- Raw synchronized sensor sessions with manifest, clock basis, calibration notes and data fields.
- Model comparison table with accuracy/quality findings, FPS and hardware environment.
- Side-by-side ablations: RGB only vs RGB + IMU-assisted processing; detection only vs tracking; depth alternatives.
- Annotated failure cases and a 5–8 page report describing architecture, assumptions, mathematics, results and next steps.

Month 2 direction: only after reliable perception and validated timestamps, consider stronger VIO/SLAM, temporal BEV, calibrated metric depth / stereo, robust multi-object velocity and risk forecasting. Not a vehicle control stack.

TECHNICAL LEARNING ADDENDUM

Added to the original 4-week autonomous-driving plan. This study track deepens Kalman filtering, VIO, and SLAM while preserving the required lane, vehicle, depth and streaming milestones. Students should study the concepts progressively during Days 4, 9, 13-15, and 16-20.

How to use this addendum across the month

• **Week 1 (2-3 h):** coordinate frames, sensor physics, timestamp alignment, and camera/IMU calibration research.

• **Week 2 (3-4 h):** geometric vision and tracking motion models; why optical flow and ego-motion are different.

• **Week 3 (8-10 h):** detailed Kalman/EKF mathematics, visual odometry, IMU integration, VIO, and SLAM. Implement the toy filters and run one VIO/SLAM demonstration.

• **Week 4 (4-6 h):** compare estimators, integrate motion cues where justified, evaluate drift and discuss why full SLAM may not be needed for a first perception prototype.

A. Frames, motion, and inertial sensing - prerequisite

• **Frames:** phone/body B, camera C, vehicle V, world W. A 3D point changes coordinates using rotation R and translation t: p_W = R_WB p_B + t_WB. Show axes with a drawing and mark which direction gravity points.

• **Gyro:** measures angular velocity with bias and noise: omega_meas = omega_true + b_g + n_g. For short dt, delta angle approx (omega_meas - b_g)dt. Explain why a small bias accumulates over time.

• **Accelerometer:** measures specific force, not world-frame position or pure acceleration. Approx a_W = R_WB(a_meas - b_a) + g_W. Explain gravity subtraction and why double integration causes drift.

• **Rotations:** compare Euler angles, rotation matrices, and unit quaternions; describe gimbal lock and why quaternions are common in navigation.

```
p_W = R_WB * p_B + t_WB ; theta[k+1] ~= theta[k] + (gyro[k]-bias)*dt
```

Exercises (required)

• Numerical: at constant measured yaw rate 0.11 rad/s with true bias 0.01 rad/s for 10 s, estimate yaw with and without bias correction; convert both to degrees.

• Code: plot 3-axis gyro, acceleration, and estimated yaw from recorded phone samples; mark stationary segments. Show one case where phone orientation differs from car heading.

B. Camera geometry before VIO

• Pinhole model: u = fx X/Z + cx and v = fy Y/Z + cy. Define focal lengths fx, fy, optical center cx, cy, and depth Z. Explain why one RGB frame cannot uniquely recover metric scale.

• Study image distortion, calibrated intrinsics K, epipolar constraint, feature matches, outlier rejection with RANSAC and recoverPose; contrast essential matrix with homography.

• Explain why static points yield image motion during camera motion, and why moving cars violate the static-scene assumption used by many VO algorithms.

KALMAN FILTERS: INTUITION + EQUATIONS

C. Start from uncertainty rather than formula memorization

• **Intuition:** prediction trusts a motion model, measurement trusts a sensor. Covariance records how uncertain each is. The Kalman gain determines how much the new observation should shift the prediction.

• **State:** x (unknown quantities), **F:** transition model, **u/B:** control/input, **P:** estimation covariance, **Q:** process-noise covariance, **z:** measurement, **H:** measurement matrix, **R:** measurement-noise covariance, **K:** Kalman gain.

```
Prediction: x_prior = F*x_prev + B*u P_prior = F*P_prev*F^T + Q
Innovation: y = z - H*x_prior S = H*P_prior*H^T + R
Gain/update: K = P_prior*H^T*S^-1 x_post = x_prior + K*y
Covariance: P_post = (I-K*H)*P_prior*(I-K*H)^T + K*R*K^T
```

• **Why Joseph-form covariance?** The last equation is numerically more robust than simply P_post=(I-KH)P_prior in floating-point implementation.

Required hand-calculation (scalar example)

• Start with estimated position x=10 m, uncertainty variance P=4 m^2; motion prediction adds Q=1 m^2. New independent position measurement z=14 m with R=9 m^2.

• Compute P_prior=5, Kalman gain K=5/(5+9)=0.3571, corrected position 11.4286 m, posterior variance approximately 3.2143 m^2. Explain why the estimate is nearer the prediction than the noisy observation.

• Repeat with R=1 and R=100. Describe what changes as the sensor becomes more or less reliable. Plot all cases.

D. Implement a 2D kinematic Kalman filter

• State x=[px, py, vx, vy]^T. Assume constant velocity over dt and derive 4x4 F with dt in position-velocity cross terms. Use position measurements for z and H; choose Q/R and show their effects.

• Simulate straight motion and turning motion, noisy and missed measurements, and variable dt. Plot ground truth, raw observations, filtered trajectory, position error, and covariance envelope.

• Explain why a constant-velocity Kalman filter is common in multi-object tracking and why it alone is not VIO or SLAM.

E. EKF versus linear KF

• For nonlinear dynamics x_next=f(x,u)+w and measurement z=h(x)+v, linearize locally using Jacobians F = df/dx and H = dh/dx; then apply analogous covariance propagation and updates.

• Show a simple heading-aware vehicle state [x,y,heading,speed] with x_next=x+v cos(heading)dt and y_next=y+v sin(heading)dt. Derive at least two non-zero Jacobian terms: dx/dheading=-v sin(heading)dt and dy/dheading=v cos(heading)dt.

• Study innovation gating, observability, covariance consistency, angle normalization and failure under wrong noise assumptions; no need to implement an industrial-strength ESKF in Month 1.

VISUAL ODOMETRY, VIO AND SLAM

F. Build the concepts in the correct order

• **Visual odometry (VO):** estimate frame-to-frame or keyframe camera motion using visual landmarks; may be feature-based or direct. Monocular VO generally has unknown metric scale.

• **Inertial odometry:** integrate gyroscope and accelerometer with gravity/bias models; useful at high rate but drifts rapidly without corrections.

• **Visual-inertial odometry (VIO):** fuse visual constraints and inertial dynamics to estimate pose, velocity and biases. Research loosely coupled vs tightly coupled approaches.

• **SLAM:** simultaneous localization and mapping; adds a consistent map and often loop closure/global optimization. Distinguish visual SLAM, visual-inertial SLAM, and LiDAR SLAM.

• **Loop closure:** recognize a previously visited location and correct accumulated trajectory drift using pose-graph optimization; explain risks from mistaken place recognition.

G. VIO state, architecture, and the mathematical intuition

```
Typical state: x = [p_WB, v_WB, q_WB, b_g, b_a, landmarks(optional)]
```

• **p:** 3D position; **v:** velocity; **q:** quaternion orientation; **b_g, b_a:** gyro and accelerometer bias; **landmarks:** scene feature locations when explicitly represented.

• **Propagation:** IMU runs many times between frames, predicting attitude, velocity and position. **Correction:** a new frame supplies reprojection/feature constraints; a filter or optimizer adjusts the pose and biases.

```
p_next ~= p + v*dt + 0.5*(R*(a_meas-b_a)+g)*dt^2
v_next ~= v + (R*(a_meas-b_a)+g)*dt
Image residual: r = observed_pixel - project(K, pose, landmark)
```

• **IMU preintegration:** summarize many short high-rate IMU readings into motion constraints between adjacent camera keyframes; discuss why it saves repeated integration during optimization.

• Compare **filter-based VIO** (e.g., MSCKF family) with **optimization-based VIO** (e.g., VINS-Mono / fixed-lag smoothing). Study their states, update timing, and computational tradeoffs.

H. SLAM back-end and failure mechanisms

• Study front-end feature detection/matching/tracking, local mapping, bundle adjustment, keyframes, loop detection, global pose graph and relocalization. Draw the dataflow from frames to optimized poses/map.

• Explain the tradeoffs of feature-based versus direct methods and why dynamic vehicles, repetitive roads, low texture, night driving, rolling shutter, blur and wrong extrinsics break pose estimation.

• Research gauge freedom, scale observability, initialization, camera-IMU extrinsic and temporal calibration. Contrast an IMU being available with a well-calibrated VIO pipeline actually working.

Concept check questions (answer without copying definitions)

• Why can image-only odometry estimate direction of motion yet fail to determine meters? Under which motions can IMU help establish scale?

• Why does integrating noisy acceleration twice make position drift much worse than integrating gyro once for orientation?

• Why does revisiting a place help SLAM but not necessarily an ordinary locally integrated VIO pipeline?

• Can a moving detected car be used as a stationary landmark? How should motion segmentation/outlier filtering address it?

RESEARCH, IMPLEMENTATION AND GRADING

I. Required mini-implementations within the existing 20 days

• **Days 4-5:** capture calibration metadata; implement a coordinate-frame diagram and plot synchronized gyro/video timelines. Explain offset, drift and frame exposure time.

• **Day 9:** inspect Kalman prediction/update used by a tracker, draw F/H/Q/R for a simple box motion model, and compare tracking with and without prediction.

• **Day 13:** hand-calculate scalar KF; code scalar and constant-velocity KF; plot tuning of Q/R. Derive a nonlinear motion Jacobian.

• **Day 14:** present 1-page VO vs VIO vs SLAM comparison and full architecture diagram; implement or run a feature-tracking + essential-matrix VO demo on recorded/static scenes; test a public VIO/SLAM dataset example separately.

• **Day 15:** plot gyro-integrated yaw, VO yaw/heading change if reliable, and their disagreement; optionally implement orientation complementary filter or EKF demonstration with assumptions clearly stated.

• **Days 16-18:** expose ego-motion as a typed optional input to road perception; show at least one RGB-only vs motion-assisted smoothing ablation and explain when motion compensation fails.

• **Days 19-20:** publish a 5-8 minute technical defense: KF calculations, VIO/SLAM conceptual architecture, results on phone recordings, observed failures, and next-month VIO milestones.

J. Evaluation rubric (additional assessment; does not replace the main project)

Area	Weight	Evidence
Sensor physics / frames / time	20%	Frame diagrams, calibration, plots
Kalman intuition / mathematics	30%	Hand example, KF/EKF code, covariance plots
VO / VIO / SLAM understanding	30%	Architecture, residuals, scaling, demo + explanation
Fusion reasoning / experiments	20%	Ablation, drift/latency and failure analysis

K. Recommended primary research sources

• Kalman filter: Welch & Bishop, *An Introduction to the Kalman Filter*; Bar-Shalom-style predict/innovation/update notation.

• Error-state navigation / preintegration: Joan Solà, *Quaternion Kinematics for the Error-State Kalman Filter*; Forster et al., *On-Manifold Preintegration for Real-Time Visual-Inertial Odometry*.

• Visual inertial: Qin, Li & Shen, *VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator*; project github.com/HKUST-Aerial-Robotics/VINS-Mono.

• Visual SLAM: Mur-Artal & Tardós, *ORB-SLAM2*; Campos et al., *ORB-SLAM3*; project github.com/UZ-SLAMLab/ORB_SLAM3.

• Dataset and calibration study: EuRoC MAV dataset; Kalibr camera-IMU calibration toolbox (github.com/ethz-asl/kalibr).

Scope boundary: full production-grade metric VIO/SLAM is not required within one month. A reasoned demonstration, correct mathematics, documented calibration constraints and careful comparison are required. Never base real braking/steering decisions on the prototype.